si, quello che interessa è la compatibilità di creazione. In pratica come verrebbe implementato il fingerprint ? A cosa servirebbero i controlli inspect essenziali ?

Per la compatibilità di creazione, il **fingerprint** può essere implementato in modo abbastanza semplice:

```
configurazione di creazione risolta
              ↓
normalizzazione
              ↓
JSON deterministico
              ↓
SHA-256
              ↓
label sul container
```

I controlli `inspect` servono anzitutto a leggere quella label, verificare l'identità e conoscere lo stato del container. Non è necessario ricostruire e confrontare da `inspect` tutta la configurazione: sarebbe una duplicazione complessa che, nel nostro caso, possiamo evitare.

1. Quale configurazione viene sottoposta a fingerprint

L'input non è il testo di `runtime.yaml`, ma la struttura finale utilizzata per creare `dev`, dopo:

- composizione del runtime di linguaggio e di progetto;
- risoluzione dei percorsi e dei riferimenti;
- lettura dei file ambiente;
- applicazione degli override ammessi;
- integrazione delle impostazioni di connessione ai servizi;
- risoluzione dell'immagine.

Un esempio semplificato:

```
my $creation_spec = {
    fingerprint_version => 1,

    image_id => 'sha256:...',

    hostname => 'dev.python.foo',
    working_dir => '/app',
```

```
    entrypoint_override => undef,
    command_override    => undef,
    user_override       => undef,

    stdin_open => JSON::PP::true,
    tty        => JSON::PP::true,

    environment => {
        APP_MODE => 'development',
    },

    mounts => [
        {
            type      => 'bind',
            source    => '/home/utente/REP/DEV/Python/foo',
            target    => '/app',
            read_only => JSON::PP::false,
        },
        {
            type      => 'volume',
            source    => 'venv.dev.python.foo',
            target    => '/app/.venv',
            read_only => JSON::PP::false,
        },
    ],

    ports => [],

    network => {
        name => 'dev-python-foo_default',
    },

    resources => {},
};
```

È uno schema interno illustrativo, non il formato definitivo.

Default dell'immagine

Se non sovrascriviamo utente, entrypoint e comando, possiamo registrare "nessun override" e includere l'ID immagine. Non dobbiamo necessariamente espandere tutti i default dell'immagine nel modello: il suo ID identifica l'immagine da cui provengono.

L'ID va ottenuto dopo l'eventuale build o pull tramite `docker image inspect`, non dedotto dal tag `dev.python`.[1]

File ambiente

Il fingerprint deve dipendere dai valori effettivi letti dai file ambiente, non soltanto dai loro percorsi.

Per esempio, modificare:

```
APP_MODE=development
```

in:

```
APP_MODE=test
```

deve produrre un fingerprint diverso anche se il nome del file rimane invariato.

2. Normalizzazione prima dell'hash

La normalizzazione rende irrilevanti le differenze puramente formali.

Elemento	Normalizzazione
Mapping	Chiavi ordinate nella serializzazione
Percorsi	Percorsi assoluti risolti secondo una politica unica
Ambiente	Mapping finale, con valori stringa
Booleani	Veri booleani JSON
Porte	Formato strutturato uniforme
Mount	Ordinamento deterministico, per esempio per destinazione
Liste semanticamente ordinate	Ordine preservato, come argv del comando

Non dobbiamo ordinare indiscriminatamente tutte le liste: cambiare l'ordine degli argomenti di un comando può cambiarne il significato.

Vanno esclusi dal fingerprint:

- commenti e formattazione YAML;
- timestamp di generazione;
- label del fingerprint stesso;
- stato running/stopped;
- contenuti del progetto e del virtualenv;
- contenuti scritti nel filesystem del container.

Quindi modificare un sorgente nel bind mount non rende incompatibile il container. Cambiare la sorgente del mount, invece, sì.

3. Implementazione Perl

Una volta costruita e normalizzata la struttura, il calcolo è breve:

```
use JSON::PP ();
use Digest::SHA qw(sha256_hex);
```

```
sub creation_fingerprint {
    my ($normalized_spec) = @_;

    my $json = JSON::PP->new
        ->canonical(1)
        ->utf8(1)
        ->encode($normalized_spec);

    return 'sha256:' . sha256_hex($json);
}
```

`canonical` serve a rendere stabile l'ordine delle chiavi dei mapping. La combinazione fra JSON canonico e digest è un approccio utilizzato anche in strumenti Perl per checksum deterministici. [2]

Il contratto di normalizzazione resta importante: la serializzazione canonica non decide, da sola, come ordinare mount o normalizzare percorsi e valori.

Il valore viene poi inserito nel Compose generato:

```
services:
  dev:
    labels:
      dev.docker-dev.managed: "true"
      dev.docker-dev.project-id: "project-123"
      dev.docker-dev.role: "dev"
      dev.docker-dev.fingerprint-version: "1"
      dev.docker-dev.runtime-hash: "sha256:..."
```

4. Il fingerprint deve essere calcolato sulla configurazione finale

Questo è il vincolo principale.

Se calcoliamo l'hash e poi un `compose.user.yaml` modifica mount o ambiente, la label non rappresenta più la configurazione realmente richiesta.

Il flusso deve quindi essere:

```
tutti i contributi e override
              ↓
modello finale risolto
              ↓
creation_spec normalizzata
              ↓
fingerprint
              ↓
aggiunta delle label gestite
              ↓
creazione del container
```

Dopo il calcolo non devono essere applicati ulteriori override alle proprietà incluse nel fingerprint.

Le label del tool devono inoltre essere riservate: un file utente non deve poter sovrascrivere `runtime-hash` o `project-id`.

5. A cosa servono precisamente i controlli `inspect`

A. Leggere l'identità e il fingerprint

Dal container recuperiamo:

```
.Config.Labels
```

e verifichiamo:

```
managed = true
project-id = progetto atteso
role = dev
fingerprint-version = versione supportata
runtime-hash = hash richiesto oggi
```

Docker inspect fornisce le informazioni dettagliate dell'oggetto, incluse quelle necessarie a ispezionare il container esistente. [3] [4]

B. Conoscere lo stato

L'hash non dice se il container è fermo o già attivo.

Lo stato determina l'azione:

```
compatibile + fermo   → start
compatibile + attivo  → attach/exec secondo richiesta
incompatibile        → richiesta di ricreazione
```

Stati particolari, come pausa o rimozione in corso, richiedono un trattamento esplicito invece di un semplice `start`.

C. Verificare l'immagine effettiva

Confrontiamo anche:

```
ID immagine del container
            contro
ID immagine attualmente risolta
```

Poiché l'ID immagine è già nell'hash richiesto, questo controllo è parzialmente ridondante. Lo terrei perché è economico e rende immediata la diagnosi:

```
Il container usa un'immagine precedente.
```

È inoltre utile per intercettare incoerenze fra label e container realmente creato.

D. Non verificare nuovamente ogni mount

Per la prima versione non confronterei sistematicamente tutti i mount, l'ambiente, le porte e le risorse tramite inspect.

Se:

- il container è stato creato dal nostro tool;
- il fingerprint è stato calcolato correttamente;
- la label non è sovrascrivibile tramite la configurazione del progetto;

il confronto dell'hash è già il controllo principale di compatibilità di creazione.

L'ispezione dettagliata dei mount può restare uno strumento diagnostico, non un requisito del normale riutilizzo.

6. Procedura di riutilizzo proposta

```
my $desired_hash = creation_fingerprint($desired_spec);

my $existing = inspect_container($container_name);

if (!$existing) {
    create_container($desired_spec, $desired_hash);
}
elsif (!belongs_to_project($existing, $project_id)) {
    fail("Container omonimo non appartenente al progetto");
}
elsif (!supported_fingerprint_version($existing)) {
    fail("Compatibilità non verificabile: versione fingerprint non supportata");
}
elsif (stored_hash($existing) ne $desired_hash) {
    require_explicit_recreation();
}
elsif (container_image_id($existing) ne $desired_spec->{image_id}) {
    fail("Incoerenza tra fingerprint e immagine del container");
}
else {
    dispatch_existing_container_by_state($existing);
}
```

È pseudocodice: in particolare, `inspect_container` deve distinguere "container assente" da "Docker non raggiungibile" o altri errori. Non ogni errore inspect significa che possiamo creare un nuovo container.

7. Hash per la decisione, snapshot per la spiegazione

L'hash permette di dire:

```
uguale / diverso
```

Non permette di spiegare quali campi siano cambiati.

Per questo è utile conservare anche una rappresentazione della configurazione di creazione nello stato del progetto, associata al container:

```
container ID
fingerprint version
fingerprint
configurazione di creazione
```

Così possiamo mostrare:

```
Configurazione cambiata:
- immagine
- porta pubblicata
- mount /datasets
```

Lo snapshot non è necessario per il confronto dell'hash, ma migliora la diagnosi. Ambiente e credenziali devono essere protetti: non vanno riportati integralmente nei log o nei messaggi di differenza.

In sintesi: il fingerprint è la firma deterministica della configurazione richiesta; `inspect` serve a recuperare quella registrata, verificare appartenenza e immagine, e scegliere l'azione in base allo stato. Non serve trasformarlo in una seconda ricostruzione completa del runtime.

⁂

1. https://docs.docker.com/reference/cli/docker/image/inspect/
2. https://metacpan.org/release/SANKO/Net-BitTorrent-v2.1.1/view/Changes.md
3. https://docs.docker.com/reference/cli/docker/inspect/
4. https://docs.docker.com/reference/cli/docker/container/inspect/
5. https://metacpan.org/release/MCETIN/AmberDB-5.25.2
6. https://metacpan.org/pod/Module::Generic::File
7. https://metacpan.org/pod/Amazon::API::Provenance
8. https://docs.docker.com/engine/manage-resources/labels/
9. https://docs.docker.com/reference/dockerfile/
10. https://docs.docker.com/engine/storage/bind-mounts/
11. https://docs.docker.com/reference/cli/docker/container/ls/
12. https://docs.docker.com/build/metadata/annotations/

13. https://docs.docker.com/dhi/explore/security-concepts/digests/

14. https://docs.docker.com/scout/explore/analysis/

15. https://metacpan.org/pod/File::Raw::Hash

16. https://metacpan.org/pod/Langertha::Skeid::Registry